Lecture 6: Tiny:Bit

🌎 What is a Microcontroller

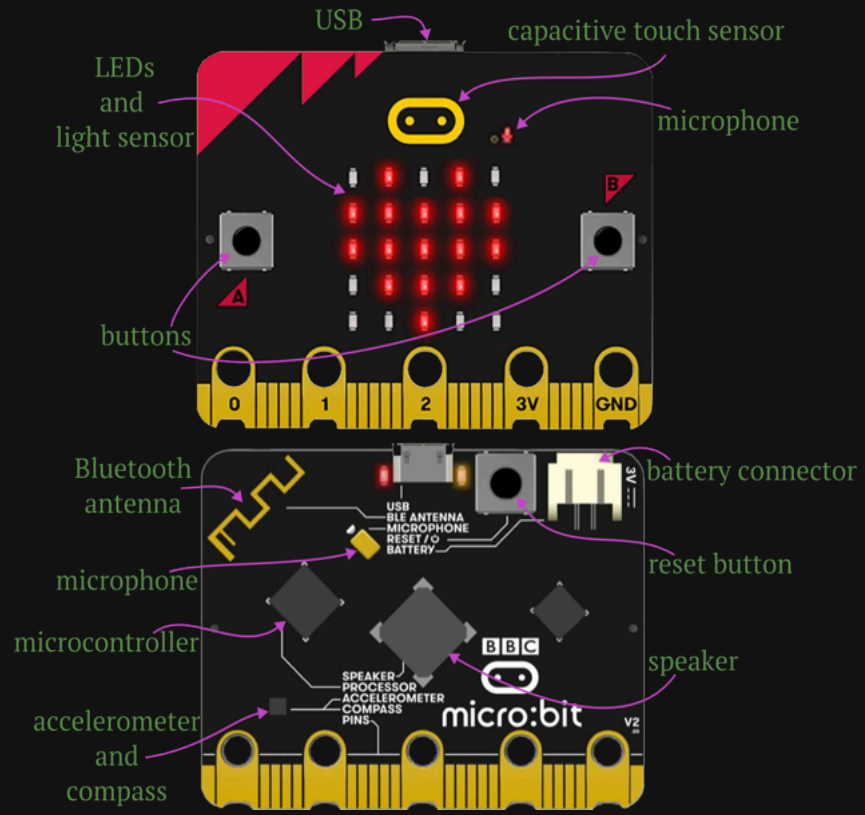
- Small, low-cost computer processor
- Low power
- Used in "embedded" applications
  - Appliances
  - Smart home devices
  - Computer peripherals
  - Cars
  - Consumer electronics
  - Medical equipment
  - Just about everywhere
- Easy to interface with sensors, displays, etc.

Microcontroller vs Microprocessor

	Microprocessor	Microcontroller
Clock Speed	~2 GHz	1-200 MHz
Number of Cores	4-32	1
Memory	2-256 GB (external)	16-512 kB (internal)
Storage	256-2,000 GB	500-2,000 kB
Power Usage	high to moderate	low to very low
Cost	high to moderate	low to very low
Peripheral Support	Complex, high bandwidth	Simple, low bandwidth

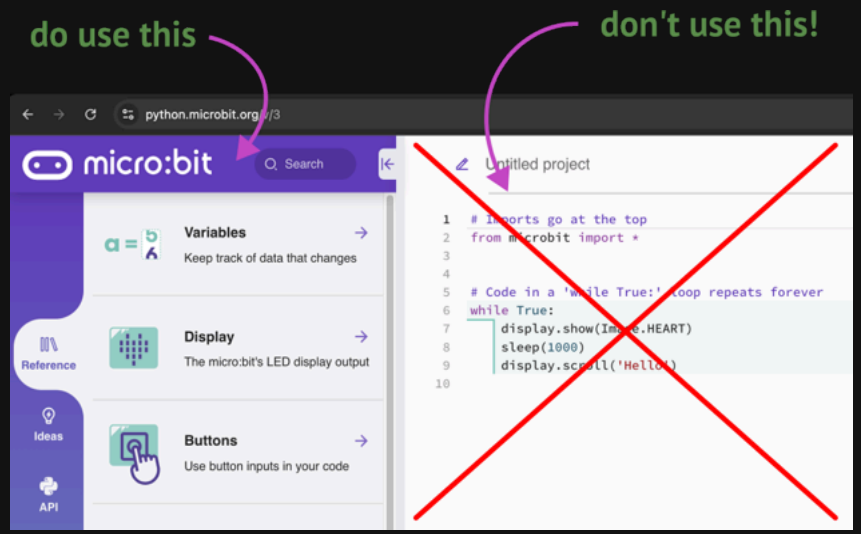
🤖 Micro:Bit

- Open-source single board computer (SBC)
- Designed by the BBC for computer education
- Has lots of cool peripherals
  - LED grid
  - Light level sensor
  - Buttons
  - Microphone
  - Speaker
  - Accelerometer (gestures, acceleration)
  - Compass
  - Bluetooth
  - USB
  - Battery connector
- Runs MicroPython!



🤖 Micro:Bit Editor

- https://python.microbit.org/v/3
- Use documentation on Micro:Bit site
- Don’t use online editor
  - Not aware of Tiny:Bit hardware
  - Will remove our Tiny:Bit firmware



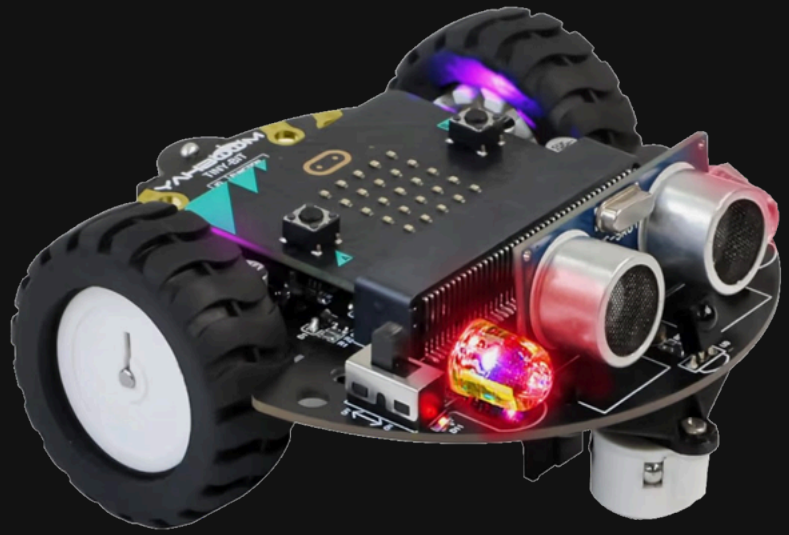
🐍 MicroPython

- A subset of Python
- Designed to run on microcontrollers
- Processor, memory, and storage constraints
- Everything we've learned so far works
- Except f-strings

```
# no f-strings in micropython
print(f"{name} has {hp} hit points")

# use string.format instead
print("{} has {} hit points".format(name, hp))

# or just use + to concatenate strings
# remember to use str() to convert non-strings
print(name + " has " + str(hp) + " hit points")
```

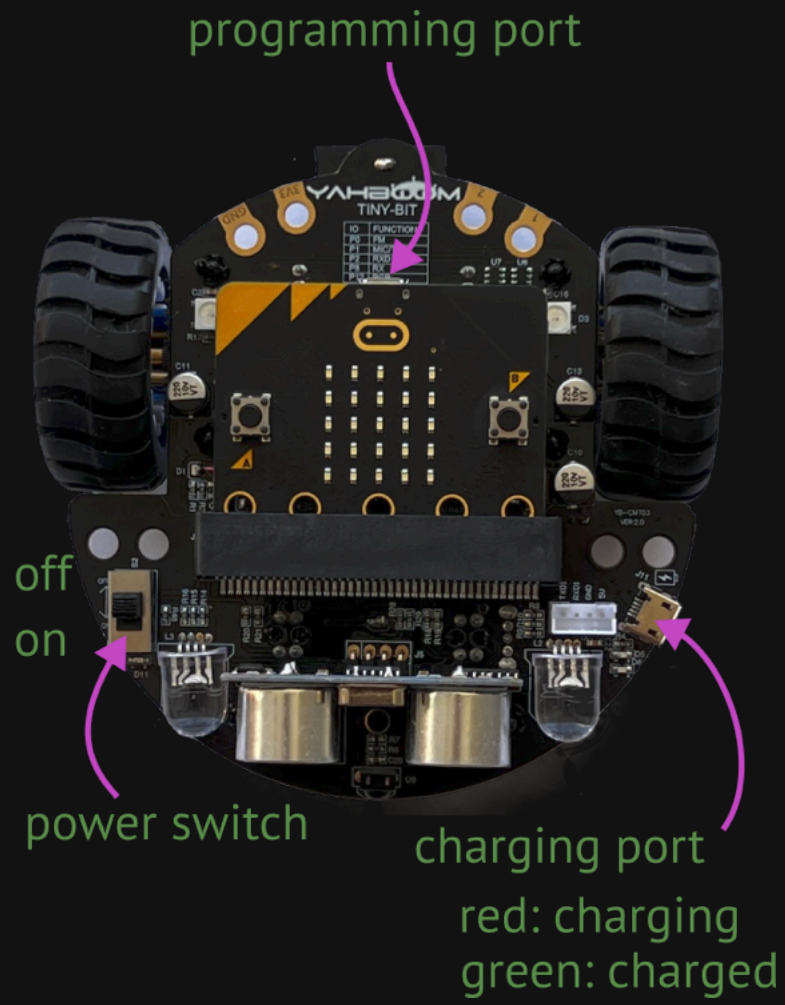
🤖 Tiny:Bit

- Motors
- RGB headlights
- RGB taillights
- Ultrasonic distance sensor
- IR receiver and remote
- Line tracking sensors
- Rechargeable battery
- Plus everything on the Micro:Bit!



🤖 Tiny:Bit Power

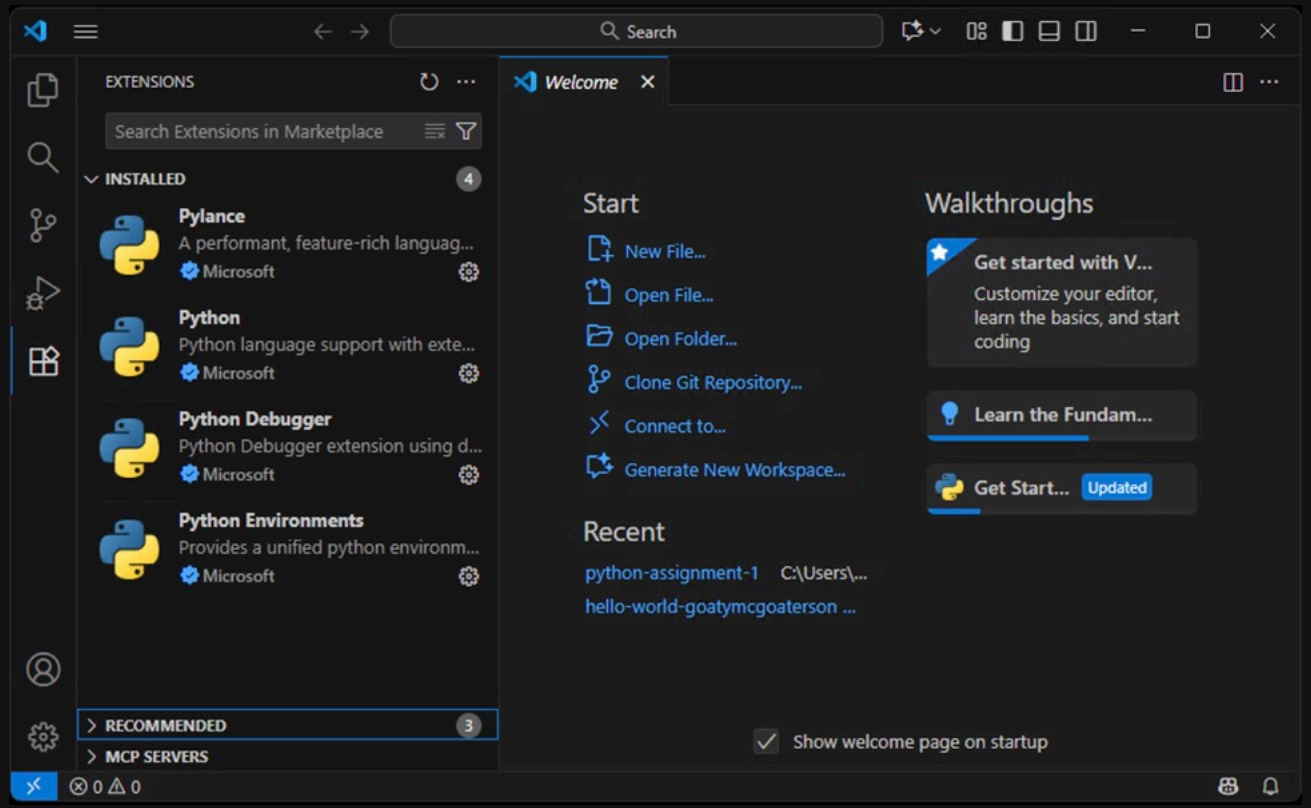
- Use charging port to charge
  - Power switch must be off
- Use Micro:Bit port for programming
  - Be careful unplugging: can brick Micro:Bit
  - *Save before programming!*
  - Your program must be called `main.py`
  - Wait 10 seconds before programming after plugging in
  - Can program with power on or off
  - Some things work poorly if power off



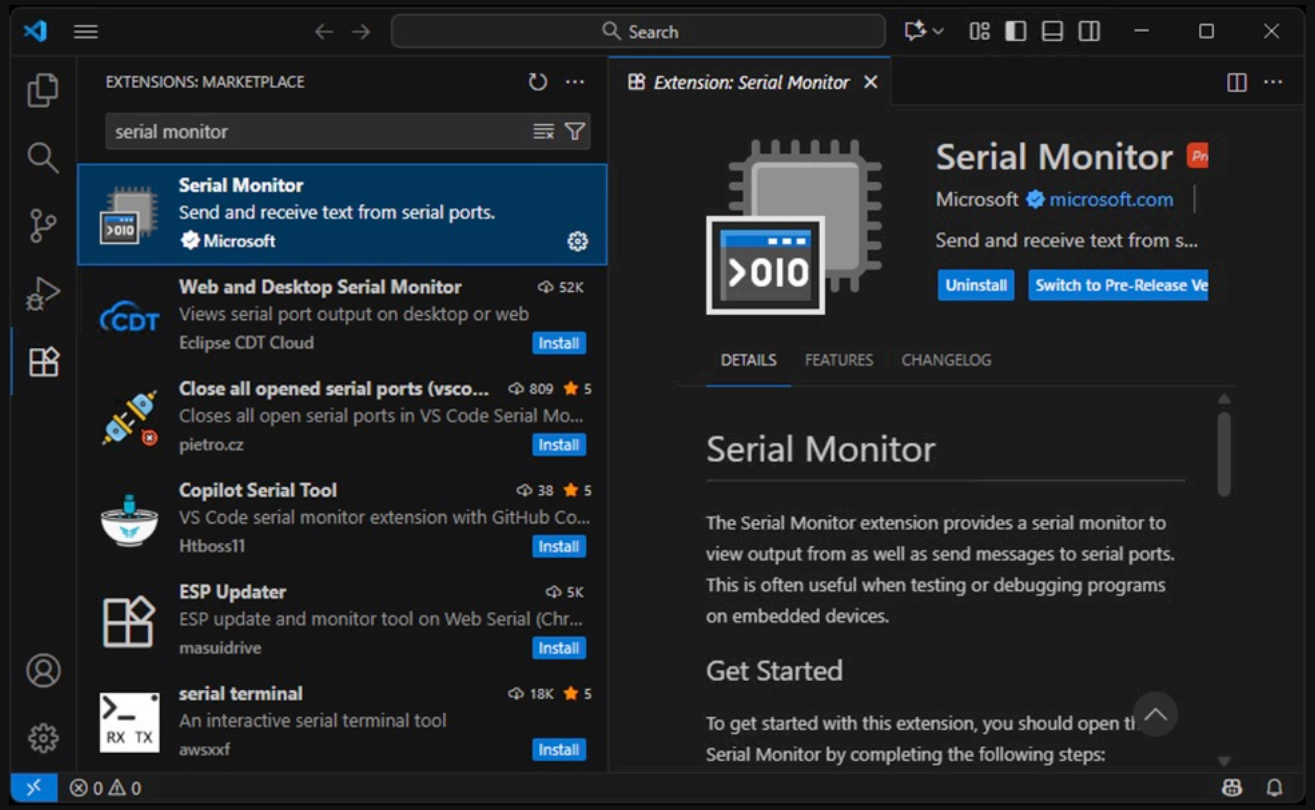
🐍 Setting Up Visual Studio Code

- Confirm all our Python extensions are installed
- Install Serial Monitor extension from Microsoft
- Create new project
- Download my `tinybit.zip` file
- Unzip it in your project directory
- Create virtual environment
- Install `microfs2` package in virtual environment
- Flash Micro:Bit with Tiny:Bit firmware
- Put `hybridge_tinybit.py` on Micro:bit
- Create simple `main.py` program
- Put `main.py` on Micro:Bit
- Celebrate! 🥳

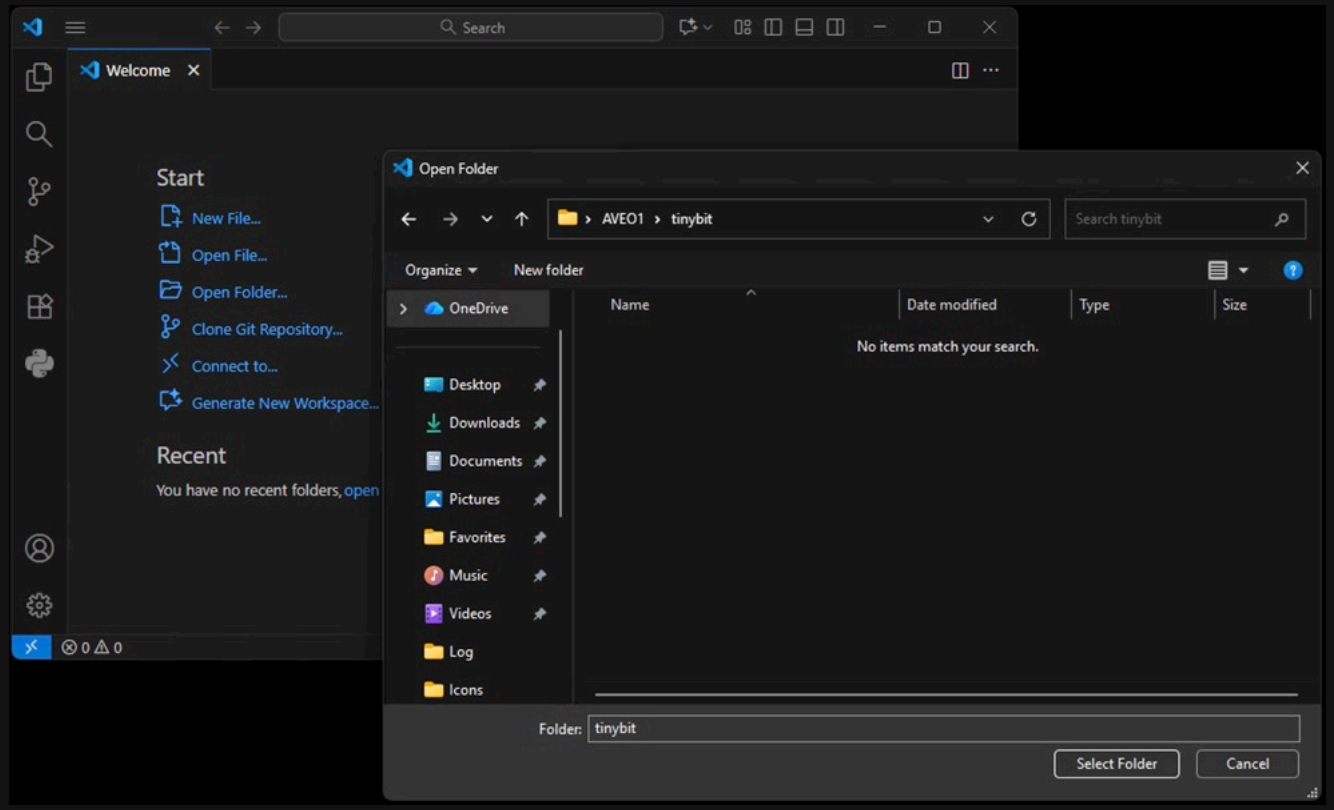
🐍 Confirm All our Python Extensions are Installed



🐍 Install Serial Monitor from Microsoft

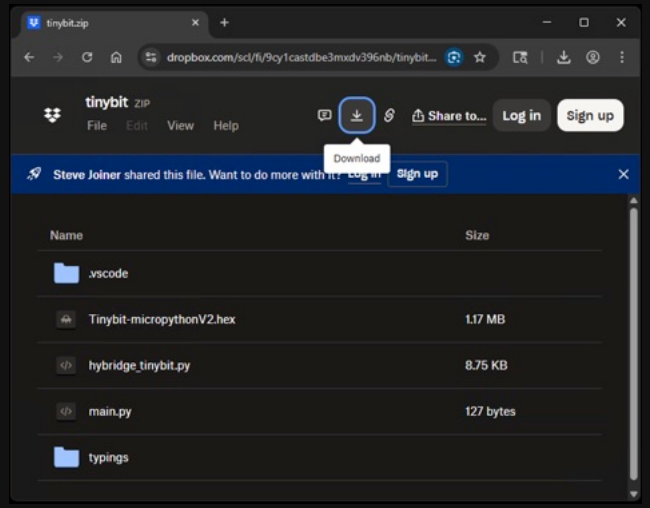


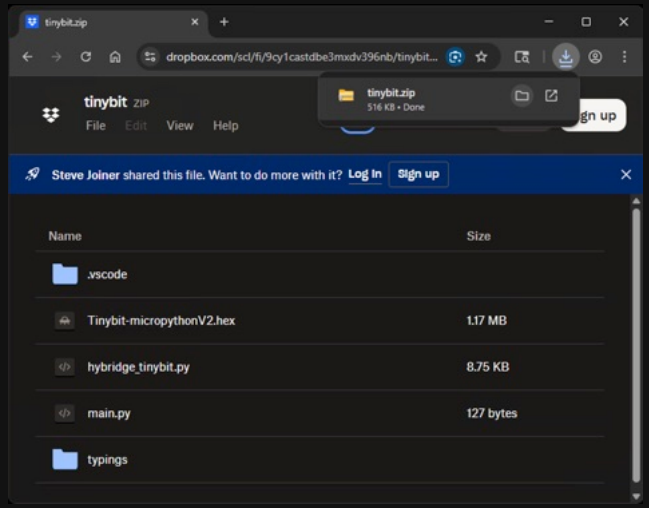
🐍 Create new Project



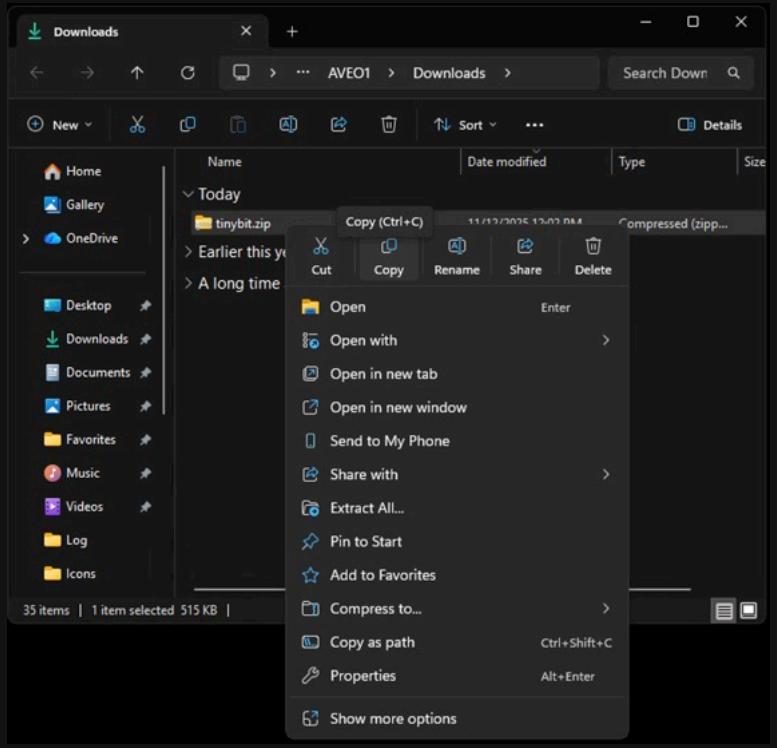
🐍 Download My `tinybit.zip` File

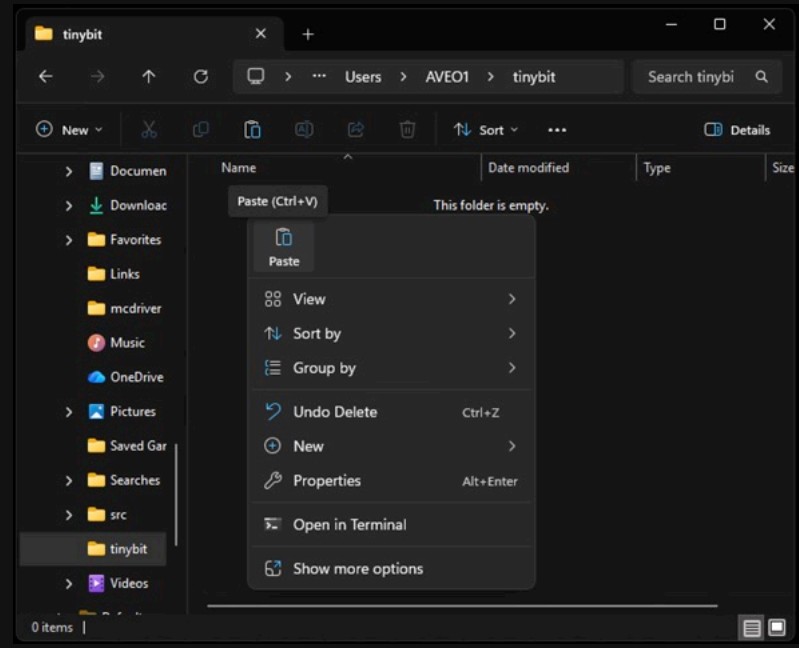
- tinybit.zip



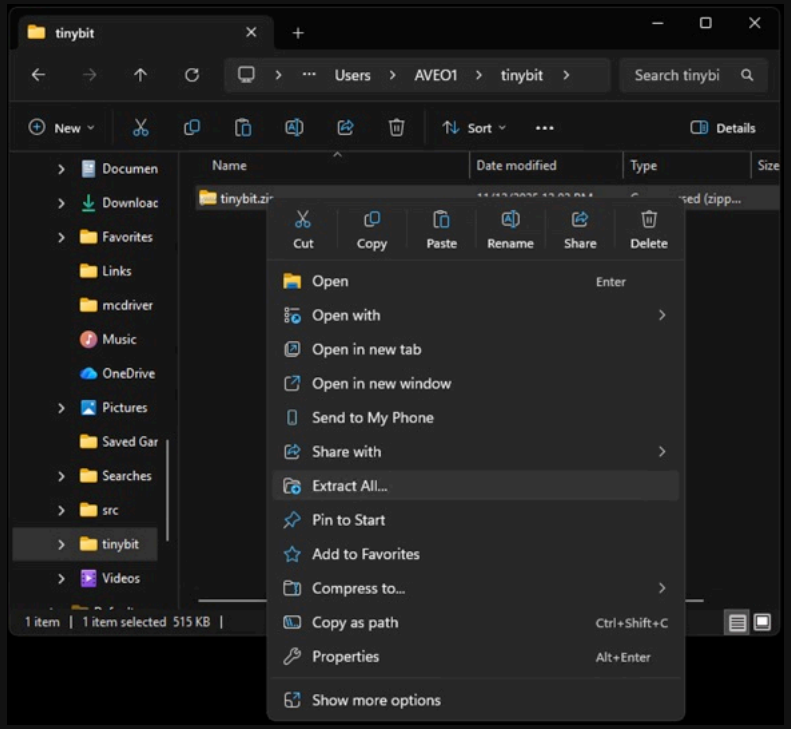


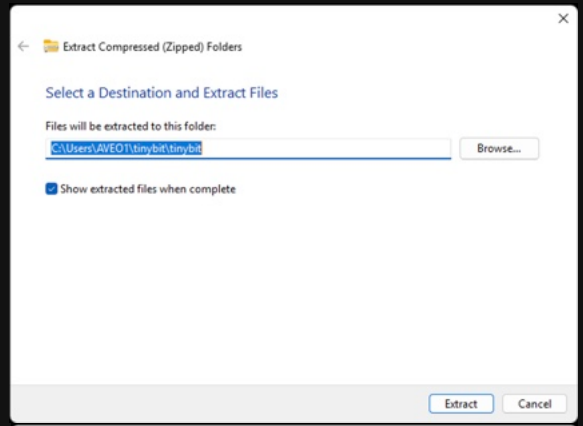
🐍 Copy `tinybit.zip` to Your Project Folder

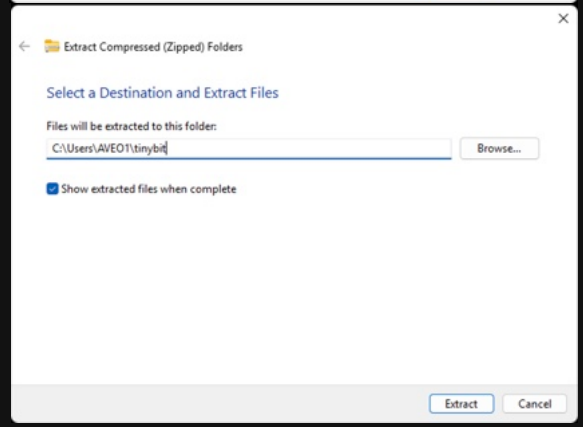




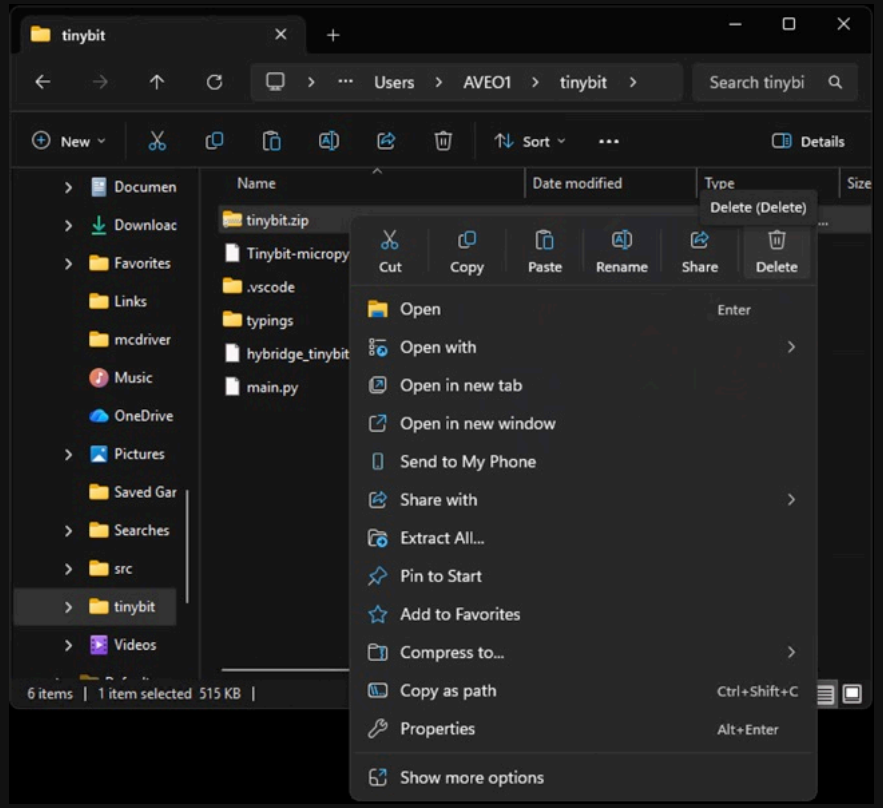
🐍 Extract `tinybit.zip` File in Your Project Folder



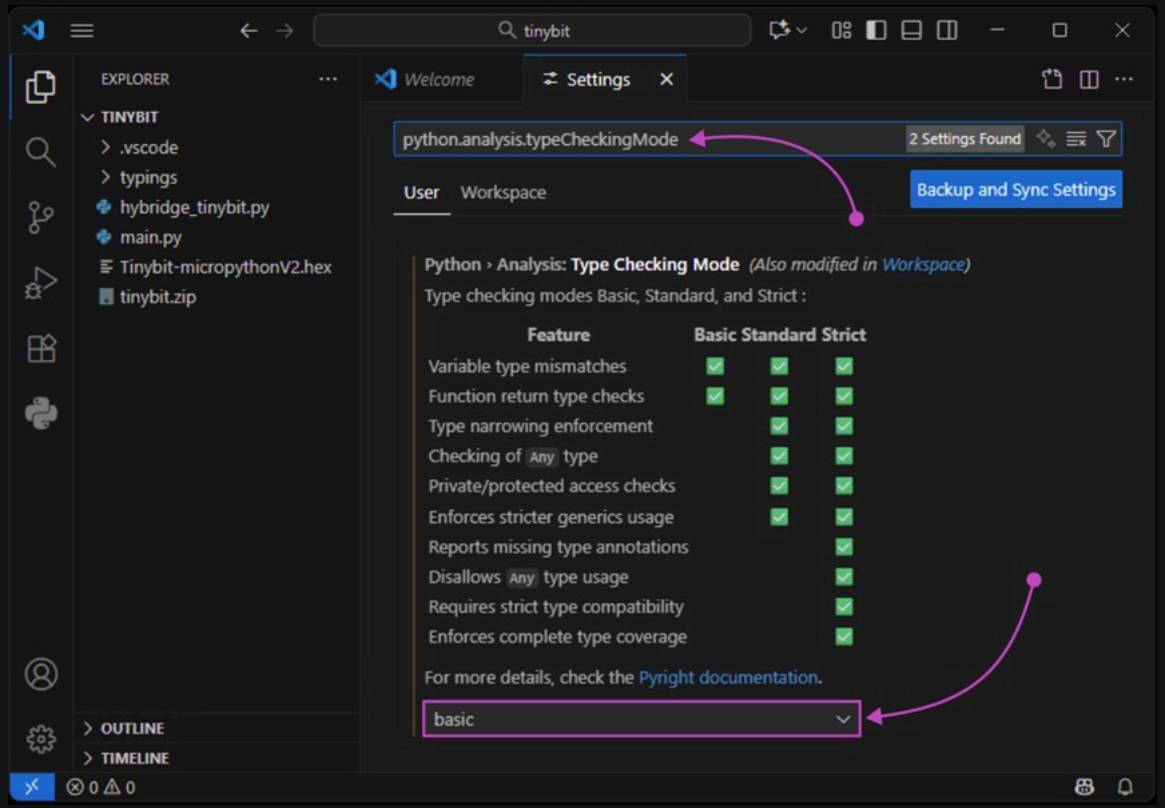




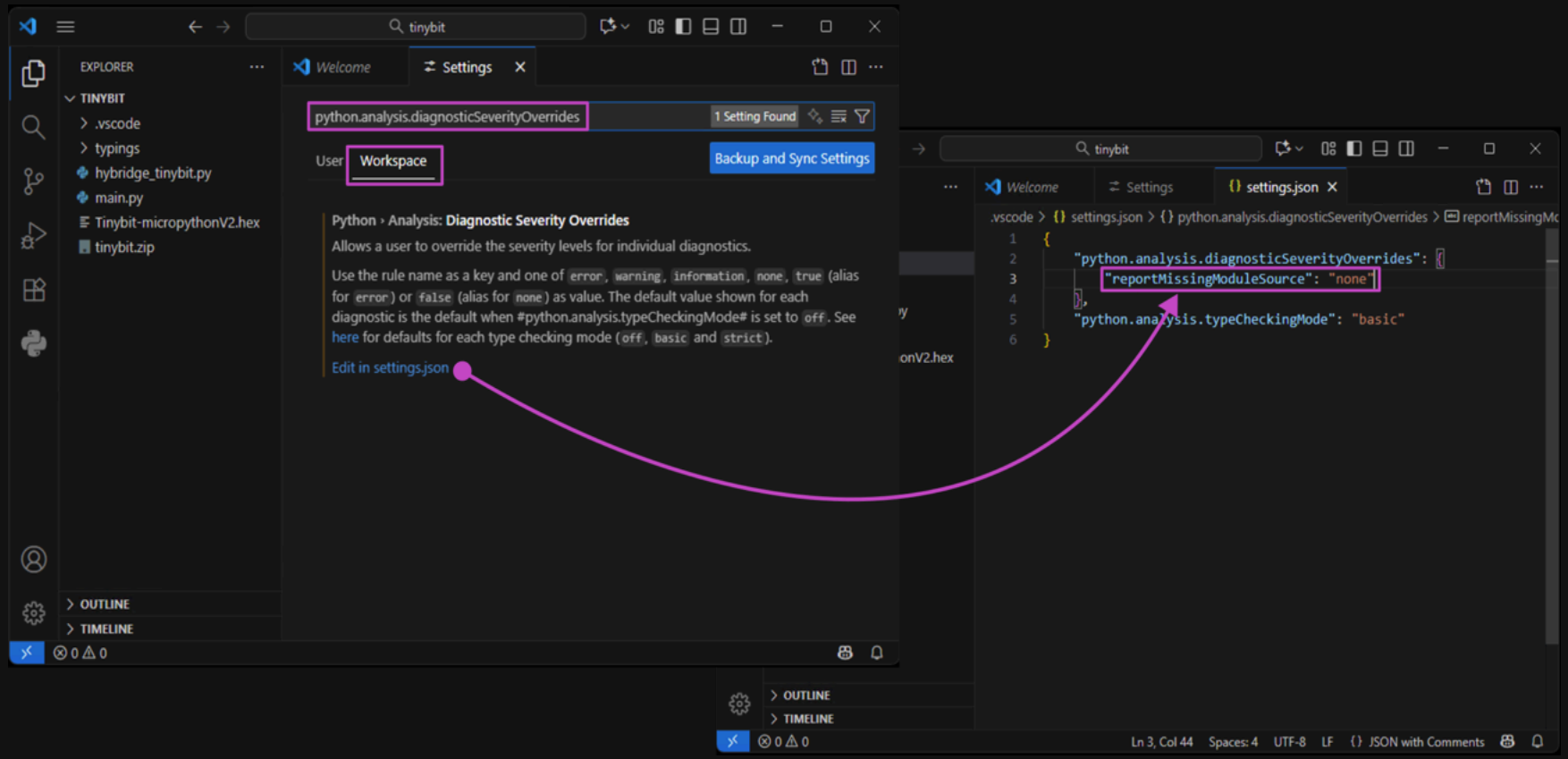
🐍 Delete `tinybit.zip` File in Your Project Folder



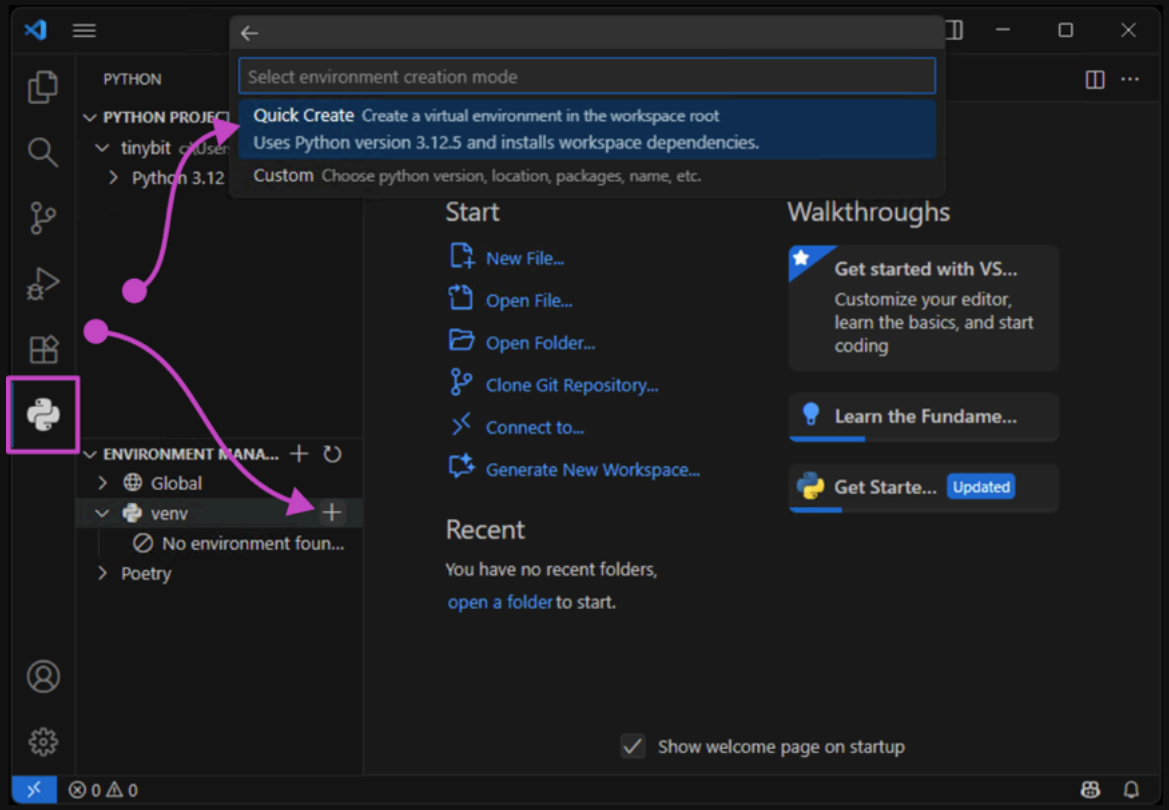
🐍 Confirm Pylance is Enabled



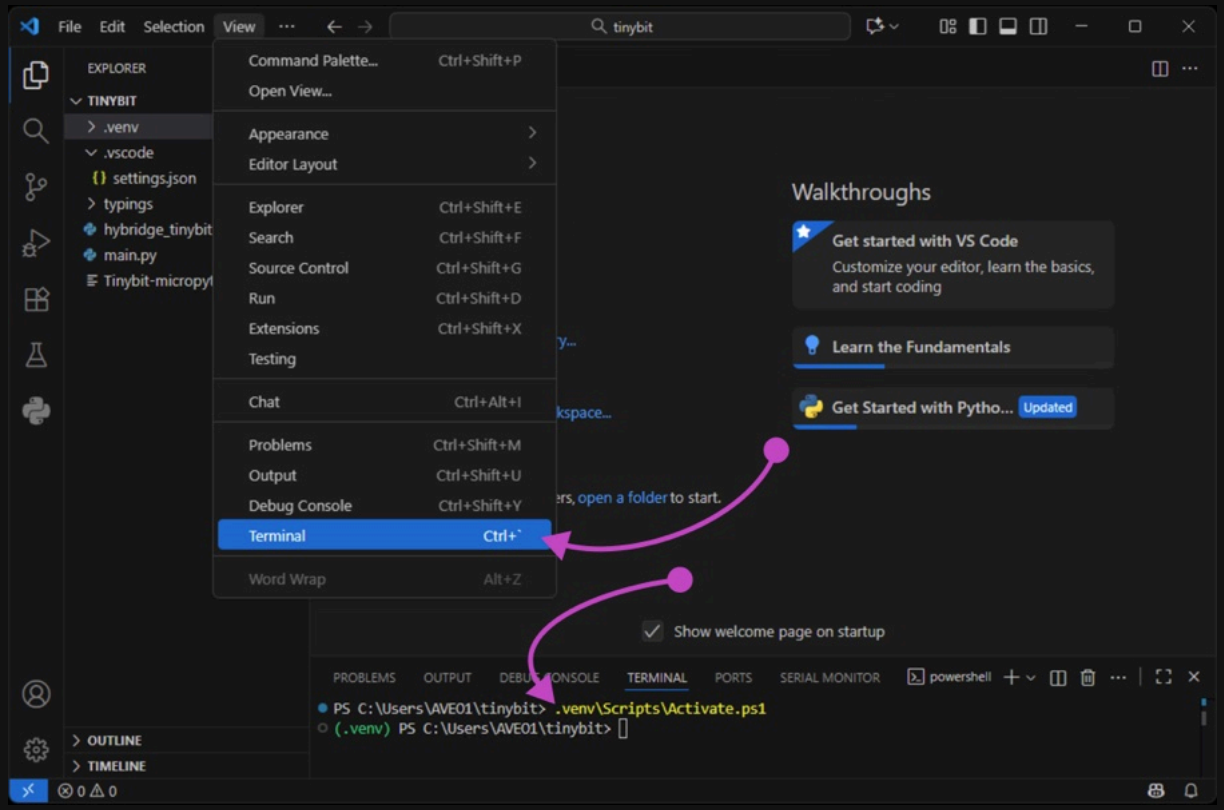
🐍 Disable `reportMissingModuleSource`



🐍 Create Virtual Environment



🐍 Activate Virtual Environment in PowerShell



🐍 Activate Virtual Environment Error

If you see this:

```
PS C:\Users\AVE01\tinybit> .venv\Scripts\Activate.ps1
.venv\Scripts\Activate.ps1 : File C:\Users\AVE01\tinybit\.venv\Scripts\Activate.ps1 cannot be loaded
because running scripts is disabled on this system. For more information, see about_Execution_Policies at
https:/go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ .venv\Scripts\Activate.ps1
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
```

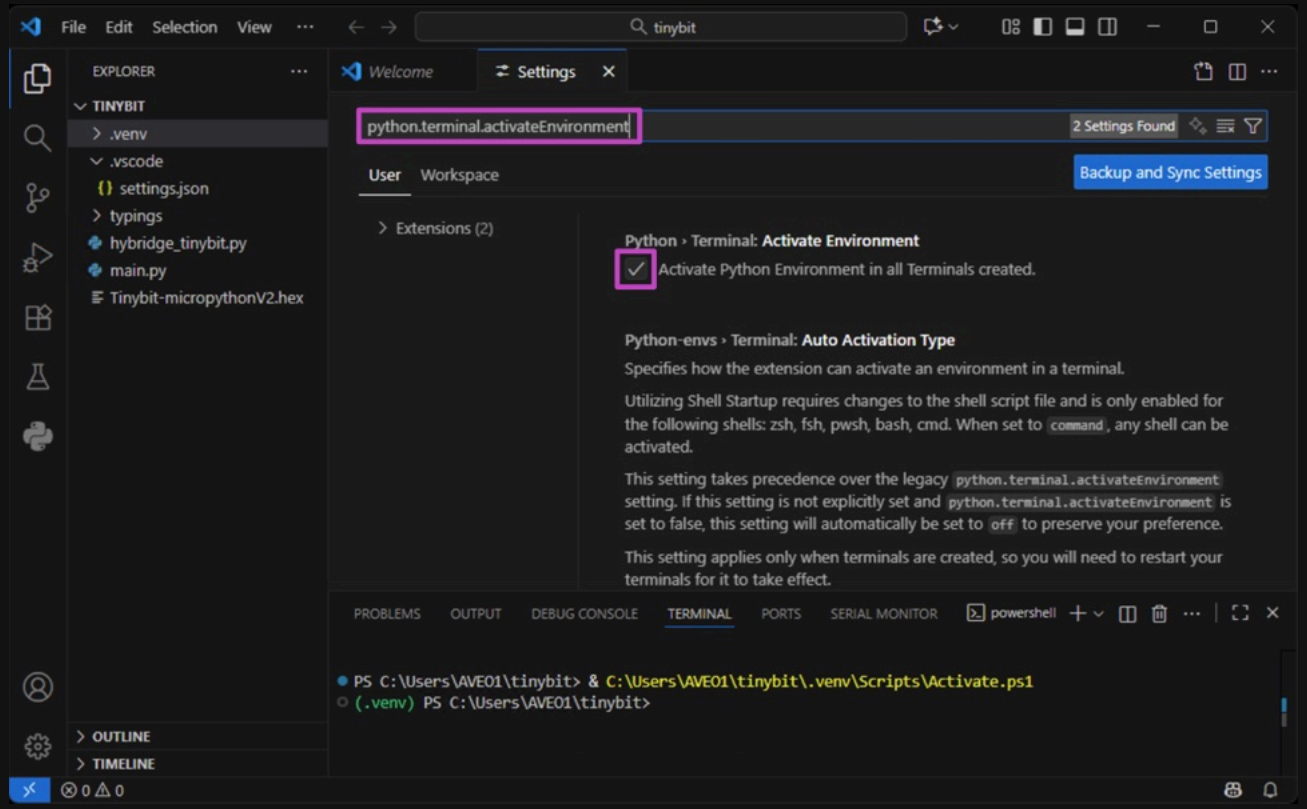
Then do this:

```
PS C:\Users\AVE01\tinybit> Set-ExecutionPolicy -ExecutionPolicy Unrestricted -Scope CurrentUser
```

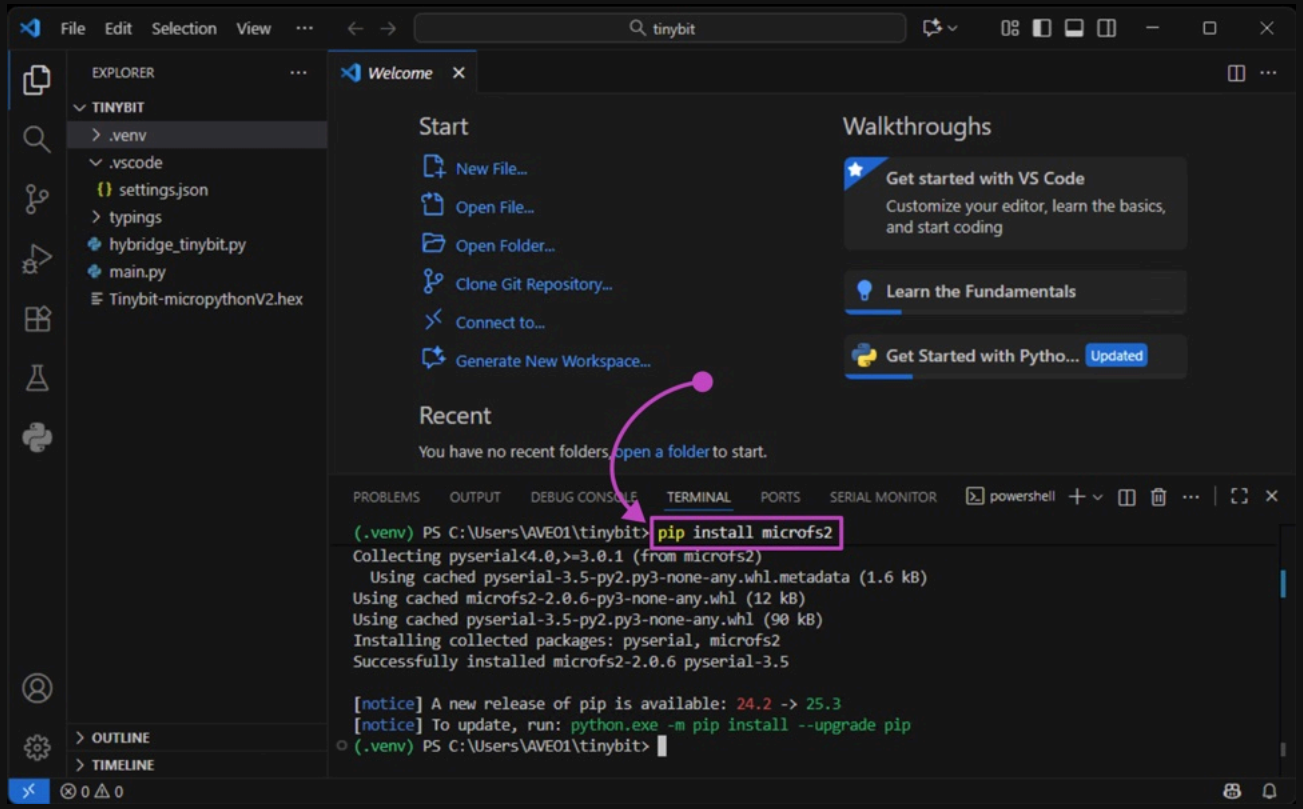
And try again:

```
PS C:\Users\AVE01\tinybit> .venv\Scripts\Activate.ps1
(.venv) PS C:\Users\AVE01\tinybit>
```

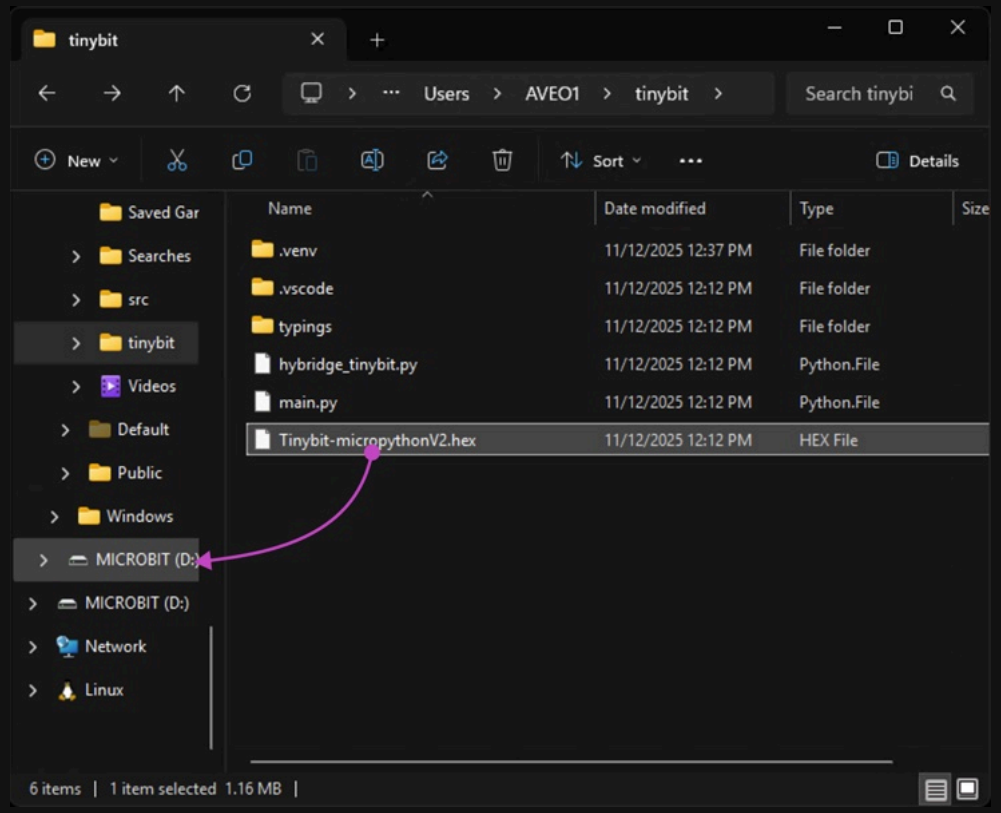
🐍 Automatically Activate Virtual Environment



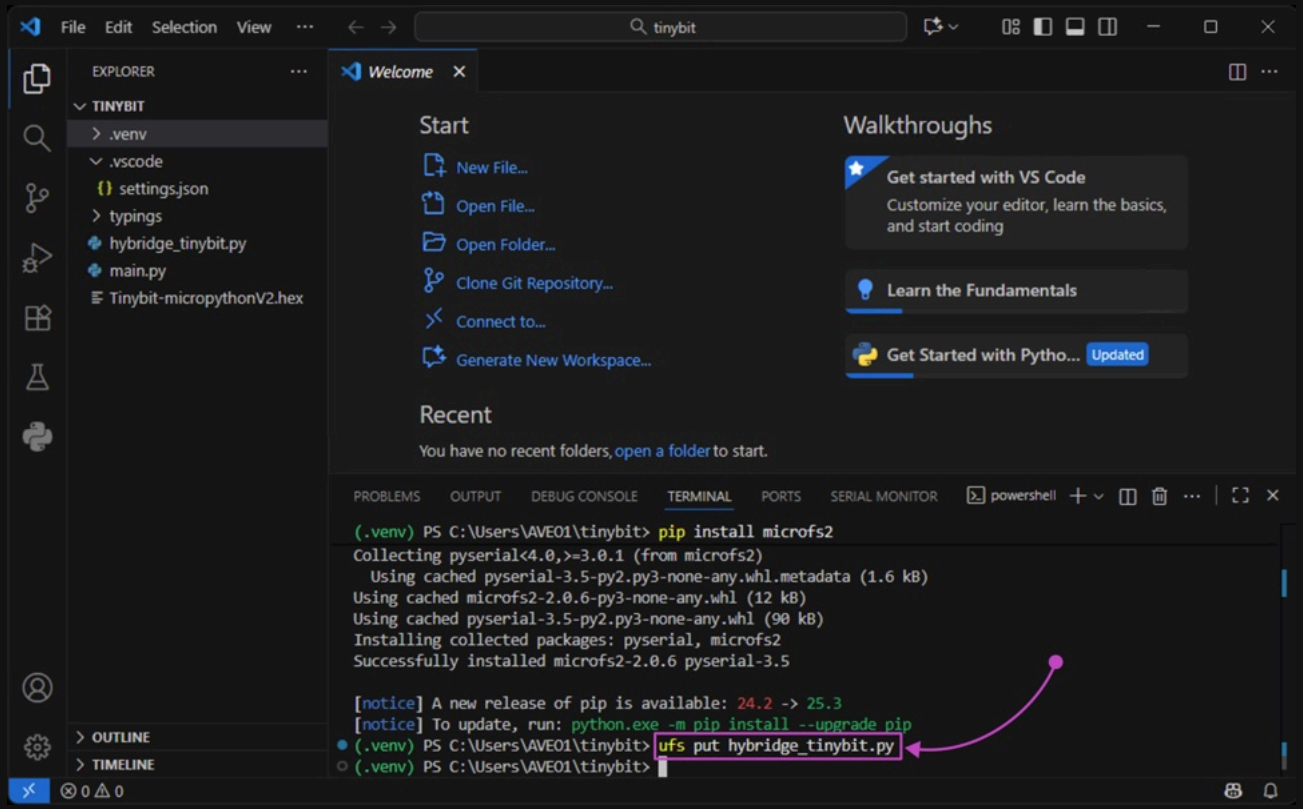
🐍 Install `microfs2` Package in Virtual Environment



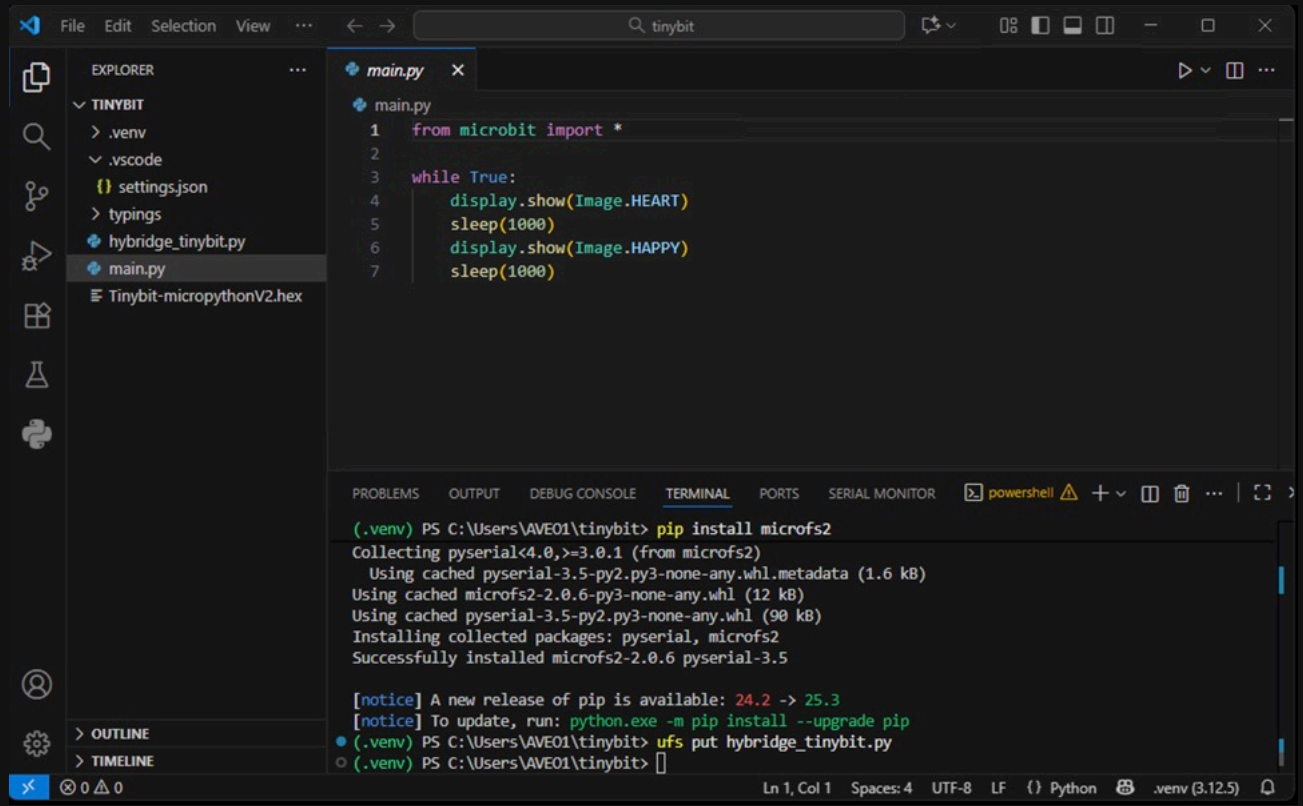
🐍 Flash Micro:Bit with Tiny:Bit Firmware



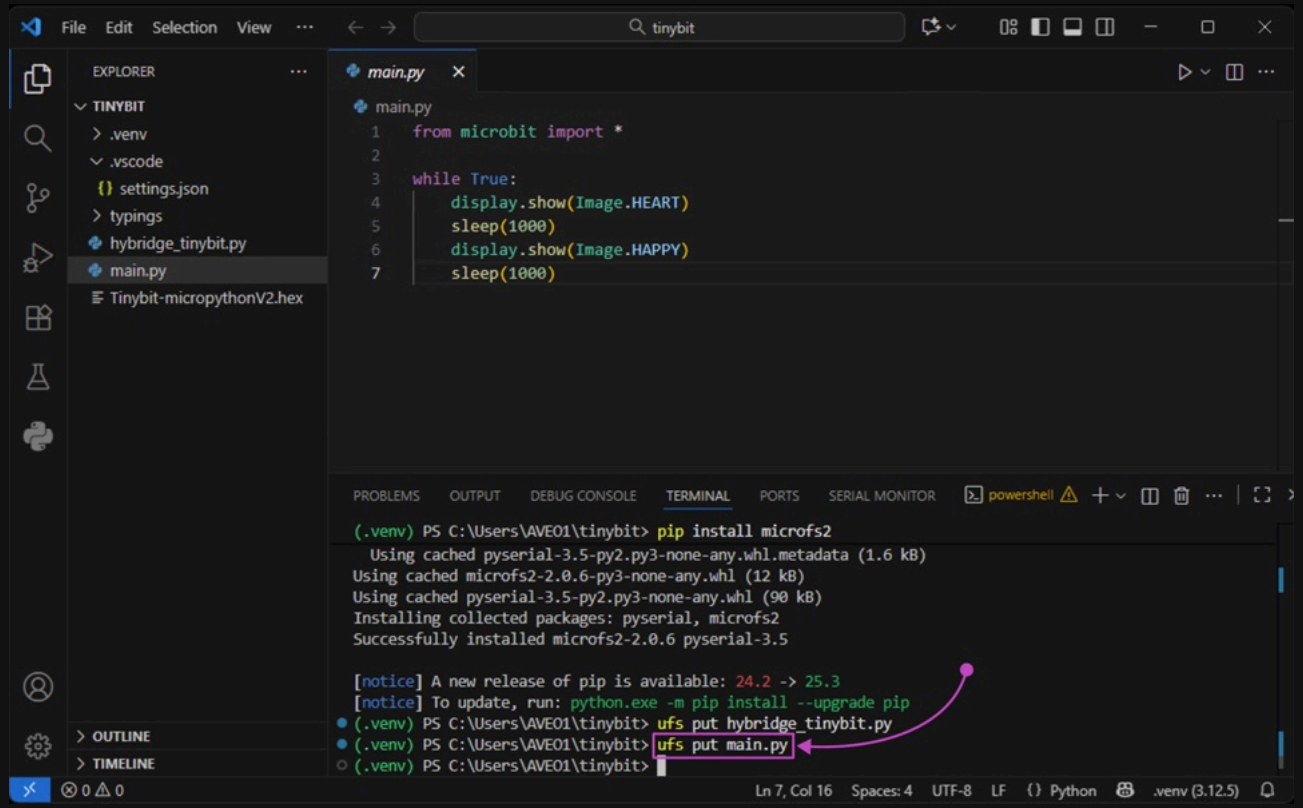
🐍 Put `hybridge_tinybit.py` on Micro:Bit



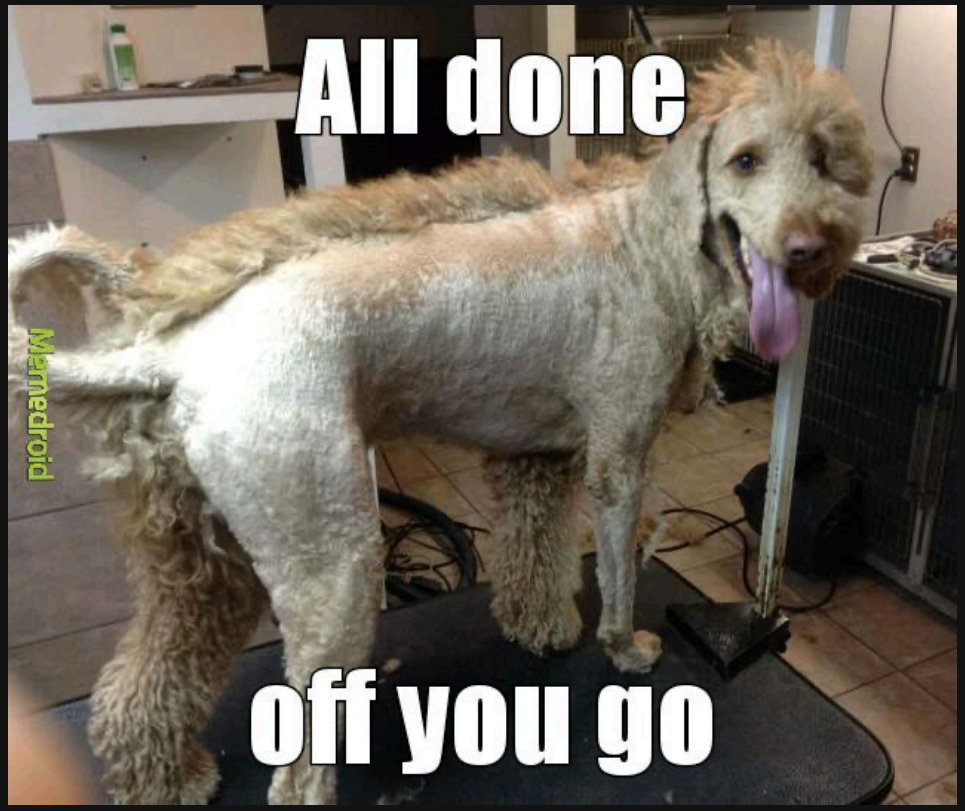
🐍 Create Simple `main.py` Program



🐍 Put `main.py` on Micro:Bit

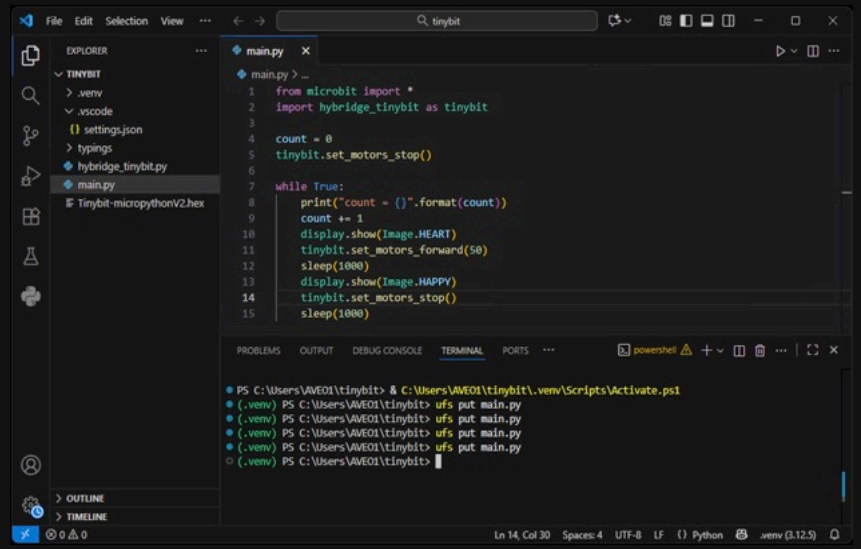


🐍 Yay!



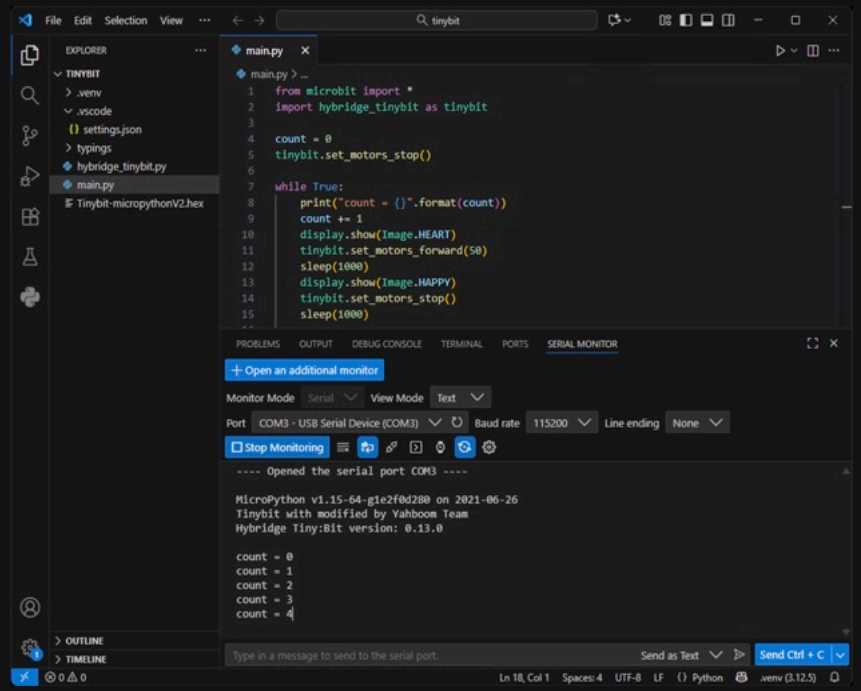
🐍 Normal Development Flow

1. Modify `main.py`
2. *Save* `main.py`
3. If you need to plug in Micro:Bit USB programming port, wait 10 seconds afterwards
4. In your venv: `ufs put main.py`



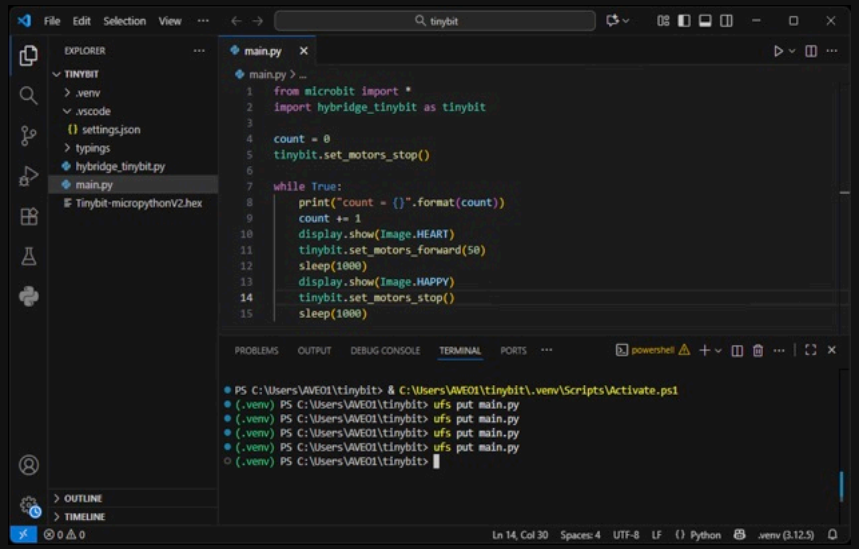
🐍 Debugging

- Option 1:
  - Micro:Bit display
  - Tiny:Bit LEDs
- Option 2:
  - Use `print`
  - View with the Serial Monitor extension
  - Have to keep USB cable attached



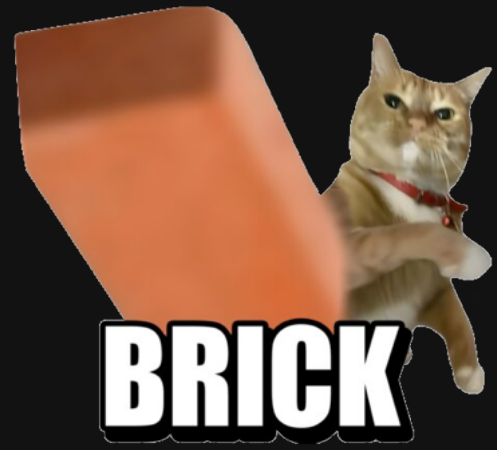
🐍 Common Problems

- Did you forget to save your `main.py`?
- Is your serial monitor connected?
  - Can't `ufs put main.py` while connected
- Are you in your virtual environment?
  - You should see `(.venv)` left of your prompt
  - If not, `.venv\Scripts\Activate.ps1`
- Is the Tiny:Bit power switch on?
- Is your battery charged?
- Did you brick your Micro:Bit?



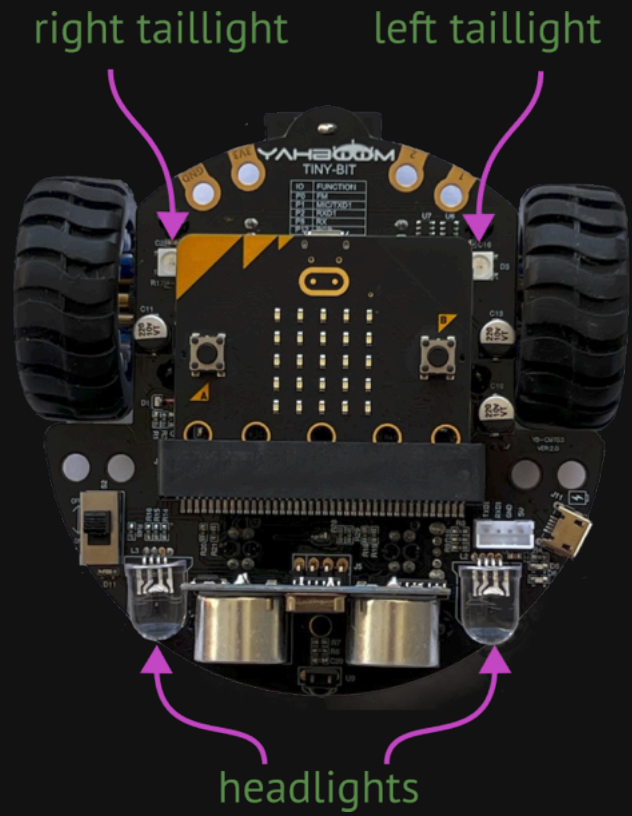
🤖 If You Brick Your Micro:Bit

1. Turn off Tiny:Bit power switch
2. Unplug Micro:Bit and wait 10 seconds
3. Plug USB into Micro:Bit programming port and wait 10 seconds
4. Copy `Tinybit-micropythonV2.hex` to Micro:Bit USB Drive
5. In your venv: `ufs put hybridge_tinybit.py`
6. In your venv: `ufs put main.py`
7. It should work now



🐍 Tiny:Bit Lights

```
import hybridge_tinybit as tinybit

color_white_full = (255, 255, 255)
color_white_half = (128, 128, 128)
color_white_quarter = (64, 64, 64)
red = (64, 0, 0)
green = (0, 64, 0)
blue = (0, 0, 64)

tinybit.set_headlights(color_white_quarter)

tinybit.set_taillight_left(blue)
tinybit.set_taillight_right(green)

tinybit.set_taillights(blue, green)
```

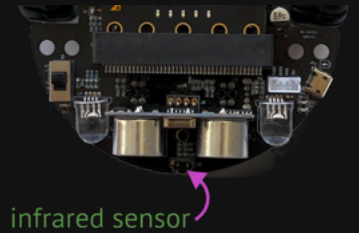


- Colors set with RGB values (0-255) as tuple or list
- Brighter drains battery faster
- I like 64

🐍 Tiny:Bit IR Remote

```
import hybridge_tinybit as tinybit

ir_code = tinybit.get_ir_code()

if ir_code == tinybit.IR_LEFT:
    tinybit.set_motors_spin_left(50)
elif ir_code == tinybit.IR_LIGHT:
    tinybit.set_headlights((64, 64, 64))
elif ir_code != tinybit.IR_NO_SIGNAL:
    print("unhandled code {}".format(ir_code))
```

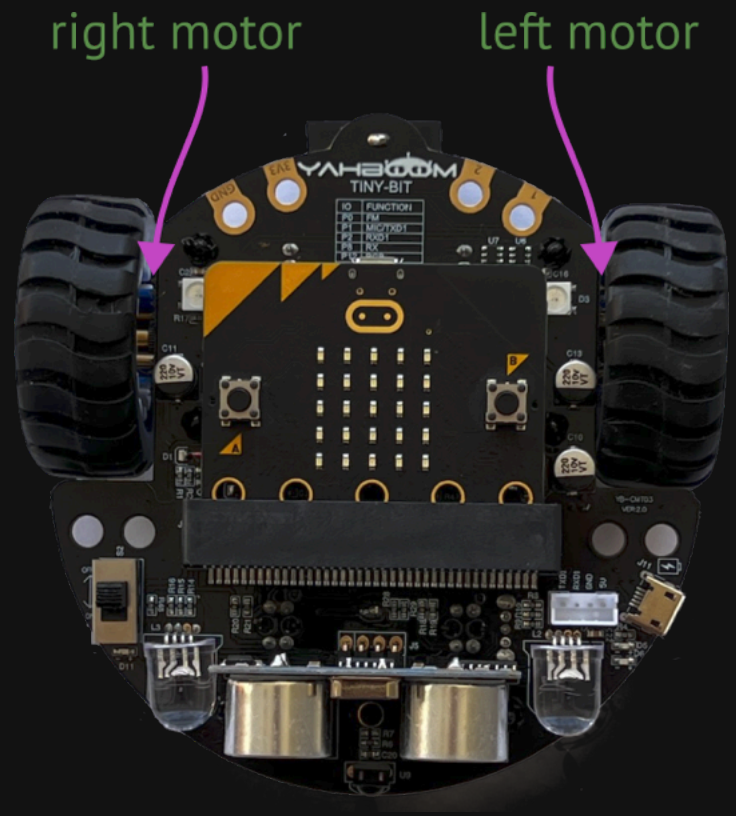
- Best reception from the front
- But kinda works from the back too
- The Micro:Bit's music interferes with the IR signal
  - Short sound effects are fine
  - Long songs cause problems
    - Codes get missed
    - Incorrect codes reported



IR_POWER	IR_UP	IR_LIGHT
IR_LEFT	IR_CENTER	IR_RIGHT
IR_BACK	IR_DOWN	IR_FORWARD
IR_PLUS	IR_0	IR_MINUS
IR_1	IR_2	IR_3
IR_4	IR_5	IR_6
IR_7	IR_8	IR_9

🐍 Tiny:Bit Motors

```
import hybridge_tinybit as tinybit

# go forward with speed 50 (0-255)
tinybit.set_motors_forward(50)

# go backward with speed 50 (0-255)
tinybit.set_motors_backward(50)

# spin left with speed 50 (0-255)
tinybit.set_motors_spin_left(50)

# spin right with speed 50 (0-255)
tinybit.set_motors_spin_right(50)

# stop motors
tinybit.set_motors_stop()
```

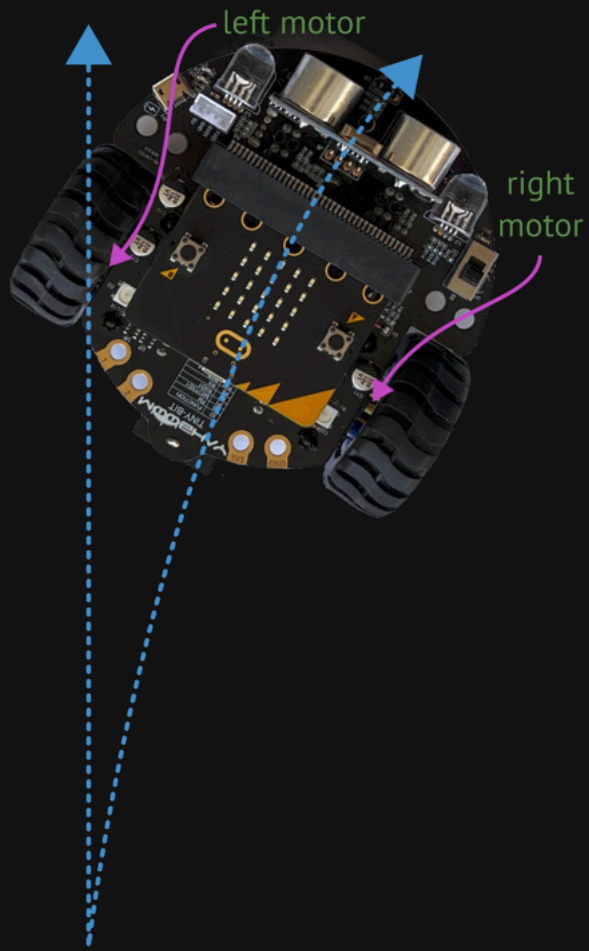


- 50 is a good speed for hard floors
- Will go slower as battery runs down
- Best battery life on hard floors
- Battery will wear down much faster on carpet

🐍 Tiny:Bit Motor Trim (Optional)

```
import hybridge_tinybit as tinybit

tinybit.set_motors_trim(3)
tinybit.set_motors_forward(50)
tinybit.set_motors_spin_left(50)
```

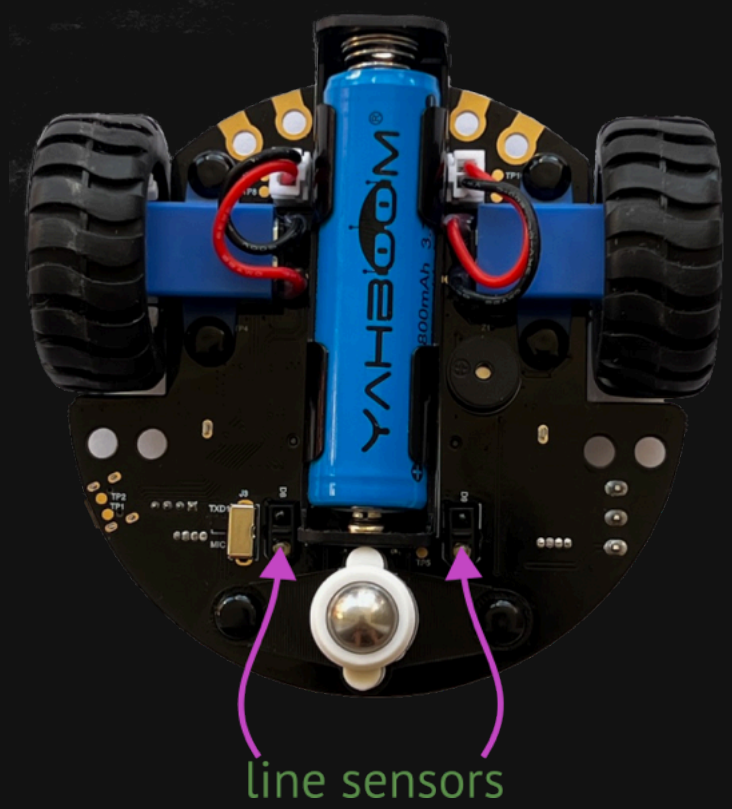
- Tiny:Bit can pull to one side
- Due to electrical and mechanical differences
- Can compensate with trim
- Positive: increase speed of left motor
- Negative: increase speed of right motor
- Set it once at top of `main.py`
- Gets applied to all other motor functions
- Drift will change as battery wears down
- Drift will changes as motors wear in



🐍 Tiny:Bit Line Sensors

```
import hybridge_tinybit as tinybit

red = (64, 0, 0)

if tinybit.has_left_track():
    tinybit.set_taillight_left(red)
elif tinybit.has_right_track():
    tinybit.set_taillight_right(red)
```

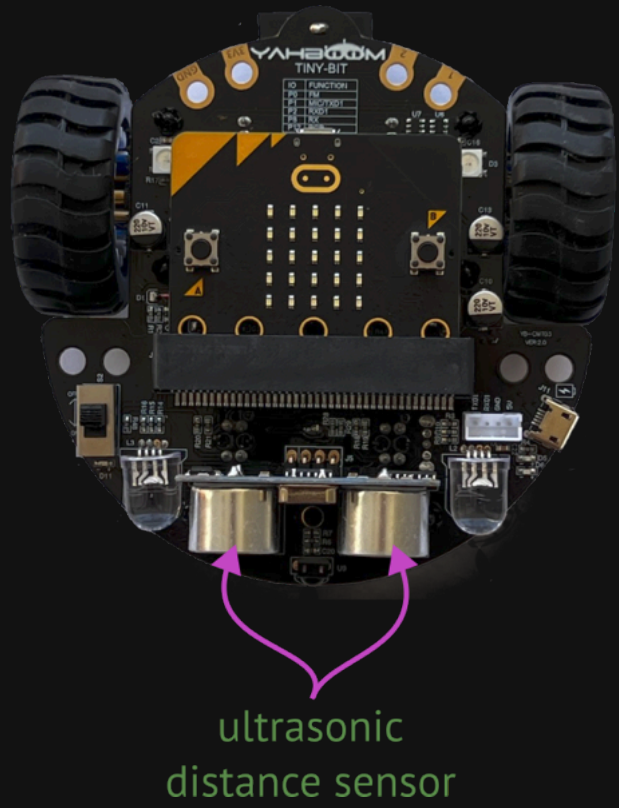
- IR emitter/detectors
- Detect black/white
  - `True` if black
  - `False` if white
- We will use for line following



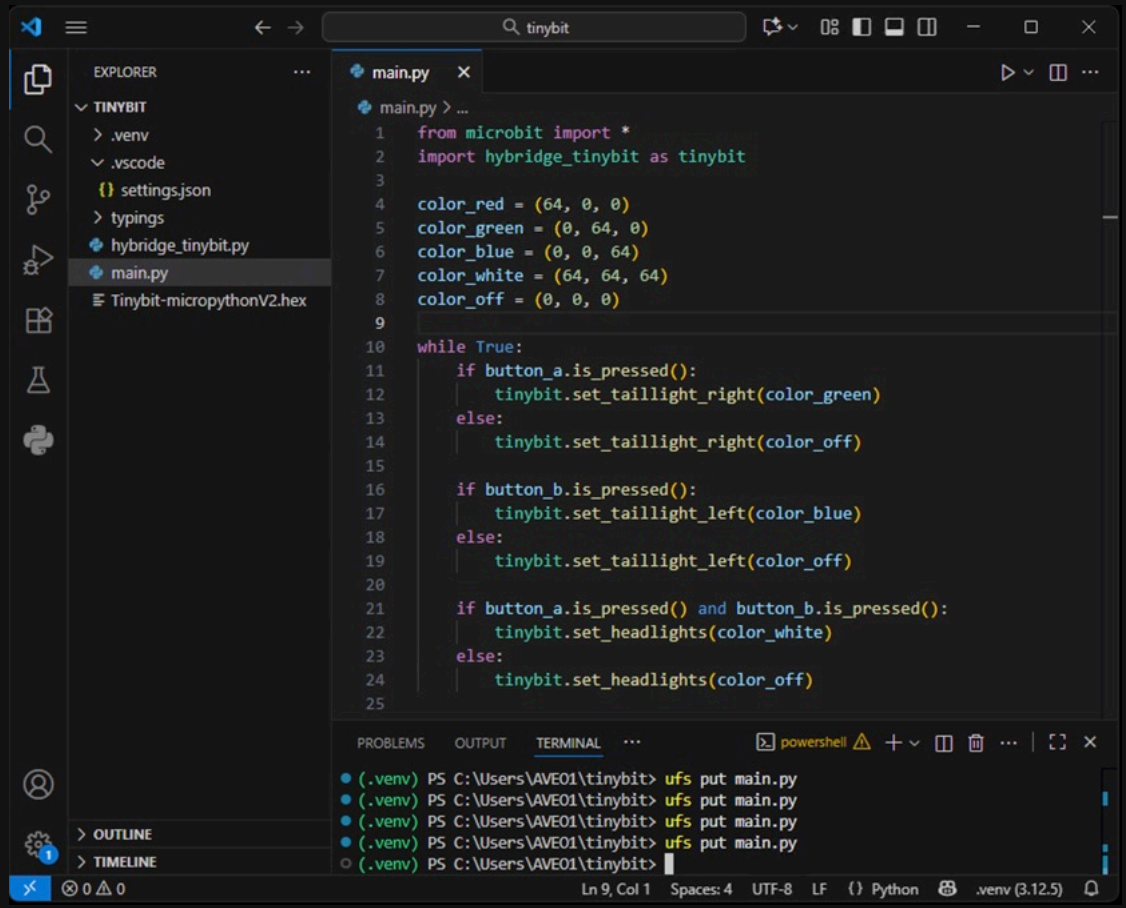
🐍 Tiny:Bit Ultrasonic Distance Sensor

```
import hybridge_tinybit as tinybit

dist = tinybit.get_sonar_distance()
```

- Returns distance (in cm)
- Best results in 10 cm - 250 cm range
- Can give erroneous results outside that range
- Blind spot around 3 cm
- ~40° field of view



🐍 Sample Program



🤖 Homework

- Control the Tiny:Bit with the IR remote
- Must at least:
  - Do things when IR remote buttons are pressed
  - Do something with the headlights
  - Do something with the taillights
  - Do something with the motors
  - Do something with the Micro:Bit display
- Have fun!
- Bring it next time to demo

- Optional ideas:
  - Play sounds with the Micro:Bit
  - Use Micro:Bit buttons
  - Detect tilt with Micro:Bit
  - Detect gestures with Micro:Bit
  - Detect light levels with Micro:Bit
  - Detect distance with Tiny:Bit ultrasonic sensor